

UJIAN AKHIR SEMESTER

Analisa dan Perancangan Berorientasi Objek

Objek Studi: Aplikasi “RAFH STUDIO”

Dosen Pengampu:

Eka Hidayat, M.Kom.

Kelas: TIF K 24A



KELOMPOK 6

Disusun oleh:

Adrian Zidan Ramadhan – 24552011224

Hafid Aliyanto – 24552011007

Muhamad Fadillah Suhada – 2455201101

Muhammad Ramdan – 24552011065

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INDUSTRI KREATIF (FIK)
UNIVERSITAS TEKNOLOGI BANDUNG 2026**

A. PERBAIKAN TERHADAP HASIL PROYEK SEBELUMNYA

Setelah aplikasi RAFH Studio selesai diimplementasikan secara nyata sebagai aplikasi Android berbasis Kotlin (package `com.example.tugas2mobile`), kelompok melakukan peninjauan ulang terhadap seluruh model UML pada dokumen tugas sebelumnya agar rancangan konsisten dengan aplikasi yang benar-benar berjalan. Beberapa class, komponen, dan struktur yang dahulu hanya bersifat asumsi/rencana pada proposal ternyata tidak diimplementasikan, sedangkan beberapa class teknis yang penting justru muncul di kode namun belum tercatat pada rancangan awal. Berikut perbaikan yang dilakukan, dikelompokkan per jenis diagram.

1) Perbaikan pada Class Diagram

- Menghapus class `User`, `Admin`, `Customer` (sebagai entitas login terpisah), `Pembayaran`, `Jadwal`, dan `Laporan` yang dirancang di proposal awal, karena aplikasi final adalah aplikasi mobile single-user tanpa sistem autentikasi/multi-role, tanpa modul pembayaran digital, dan tanpa modul cetak laporan.
- Menggabungkan konsep `BookingStudio` dan `RentalAlatMusik` pada rancangan awal menjadi satu class `Booking`, karena pada implementasi nyata satu transaksi/keranjang dapat memuat satu studio dan beberapa alat musik sekaligus — bukan dua modul yang berjalan sendiri-sendiri seperti asumsi awal.
- Menambahkan class yang benar-benar ada pada kode program tetapi belum tercatat sebelumnya: `CartManager` (pengelola status keranjang, berperan sebagai control/singleton), `DbHelper` (akses SQLite lokal), `BookingRecord` (baris riwayat transaksi), serta class antarmuka `MainActivity`, `CartActivity`, dan `HistoryActivity`.
- Melengkapi setiap atribut dengan tipe data konkret sesuai variabel pada kode, misalnya `pricePerHour: Int`, `stock: Int`, `durasi: Int`, dan `total: Int` — pada dokumen sebelumnya atribut hanya dituliskan sebagai nama tanpa tipe data yang jelas.

2) Perbaikan pada Object Diagram

Rancangan awal berasumsi adanya objek `Customer/Admin` yang login ke sistem. Object diagram diperbarui agar merepresentasikan objek nyata pada saat runtime, yaitu satu objek singleton `CartManager` yang memegang satu objek `Booking` aktif, yang di dalamnya berisi referensi ke satu objek `Studio` (opsional) dan beberapa objek `Instrument` beserta jumlah (`qty`) masing-masing.

3) Perbaikan pada Component Diagram

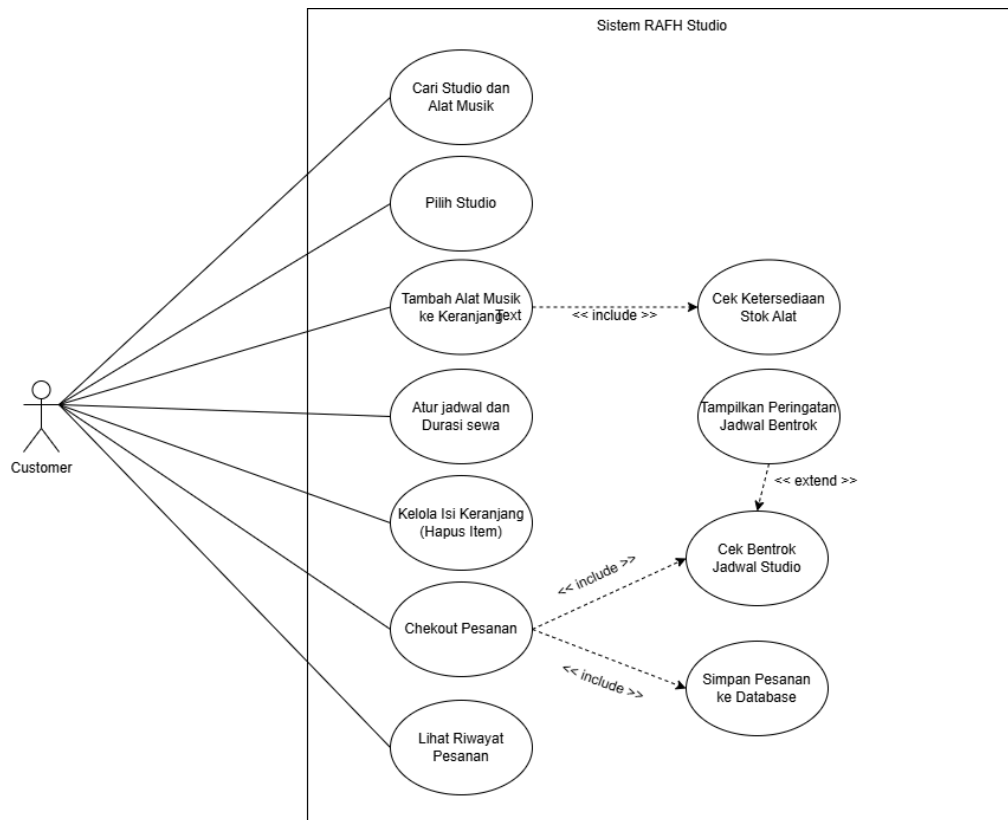
- Component diagram disederhanakan agar sesuai arsitektur nyata aplikasi Android, terdiri dari: UI Component (Activities & Adapters — MainActivity, CartActivity, HistoryActivity, CardAdapter, CartAdapter, HistoryAdapter), Control/Domain Component (CartManager, Booking), dan Persistence Component (DbHelper beserta file database SQLite rafh_studio.db).
- Login Component, Payment Component, dan Report Component pada rancangan awal dihapus karena tidak pernah diimplementasikan pada aplikasi akhir.

4) Perbaikan pada Deployment Diagram

- Rancangan awal berasumsi adanya server pusat/backend terpisah untuk menyimpan data. Pada aplikasi akhir, seluruh proses berjalan secara lokal (on-device) pada satu perangkat Android, dengan database SQLite tersimpan sebagai file lokal di penyimpanan internal perangkat — bukan pada server jarak jauh.
- Deployment diagram diperbarui menjadi satu device node “Android Device” yang menjalankan artifact app-release.apk beserta embedded database rafh_studio.db, tanpa node server maupun koneksi jaringan eksternal.

B. USE CASE DIAGRAM KESELURUHAN SISTEM

Berdasarkan aplikasi RAFH Studio yang telah diimplementasikan, sistem hanya memiliki satu aktor utama yaitu Pelanggan (pengguna aplikasi), karena aplikasi tidak memiliki mekanisme login/multi-role — seluruh pengguna mengakses fitur yang sama secara langsung. Diagram berikut menampilkan seluruh aktor, system boundary, tujuh use case utama, serta relasi include dan extend yang diperlukan.



Gambar 2.1 Use Case Diagram Sistem RAFH Studio

1) Daftar Use Case Utama

- Cari Studio & Alat Musik — pelanggan mencari/menyaring katalog studio maupun alat musik berdasarkan kata kunci.
- Pilih Studio — pelanggan memilih satu ruang studio untuk dimasukkan ke keranjang.
- Tambah Alat Musik ke Keranjang — pelanggan menambah alat musik ke keranjang (include: Cek Ketersediaan Stok).
- Atur Jadwal & Durasi Sewa — pelanggan memilih tanggal, jam mulai, dan durasi sewa (dalam jam).

- Kelola Isi Keranjang — pelanggan dapat menghapus studio atau alat musik dari keranjang.
- Checkout Pesanan — pelanggan menyelesaikan transaksi (include: Cek Bentrok Jadwal, Simpan Pesanan ke Database; extend: Tampilkan Peringatan Jadwal Bentrok).
- Lihat Riwayat Pemesanan — pelanggan melihat daftar transaksi yang pernah dilakukan.

2) Use Case Narrative (Checkout Pesanan)

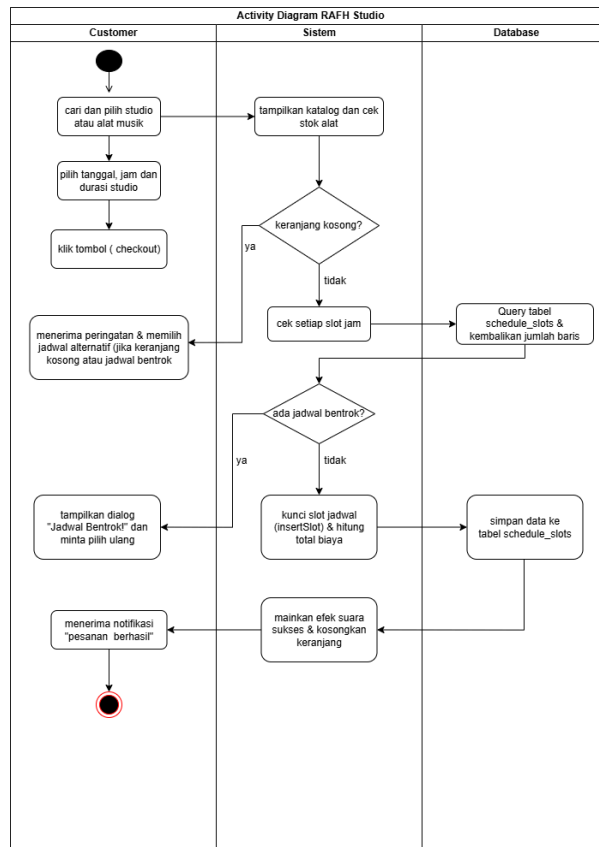
Nama Use Case	Checkout Pesanan
Aktor	Pelanggan
Deskripsi	Pelanggan menyelesaikan proses sewa studio dan/atau alat musik yang ada di keranjang menjadi sebuah transaksi tersimpan.
Precondition	Keranjang berisi minimal satu item (studio dan/atau alat musik).
Trigger	Pelanggan menekan tombol “Checkout” pada halaman keranjang.
Main Flow	1. Pelanggan memilih tanggal, jam mulai, dan durasi sewa. 2. Pelanggan menekan tombol “Checkout”. 3. Sistem memeriksa setiap slot jam yang diminta ke tabel <code>schedule_slots</code> (Cek Bentrok Jadwal). 4. Jika tidak ada bentrok, sistem mengunci slot jadwal, menghitung total biaya, dan menyimpan data pesanan ke tabel <code>bookings</code> (Simpan Pesanan ke Database). 5. Sistem memainkan efek suara sukses, menampilkan dialog “Pesanan Berhasil”, dan mengosongkan keranjang.
Alternate Flow	3a. Jika salah satu slot jam sudah terpakai, sistem membatalkan proses, menampilkan dialog “Jadwal Bentrok!”, dan tetap mempertahankan isi keranjang agar pelanggan dapat memilih jadwal lain.
Postcondition	Data pesanan baru tersimpan pada tabel <code>bookings</code> dan slot jadwal terkait terkunci pada tabel <code>schedule_slots</code> .

3) Use Case Narrative (Tambah Alat Musik ke Keranjang)

Nama Use Case	Tambah Alat Musik ke Keranjang
Aktor	Pelanggan
Deskripsi	Pelanggan menambahkan satu unit alat musik pilihannya ke dalam keranjang sewa yang sedang berjalan.
Precondition	Pelanggan sedang berada pada menu “Tambahan Alat Musik”.
Trigger	Pelanggan menekan tombol “+” pada kartu alat musik.
Main Flow	1. Pelanggan menekan tombol tambah pada salah satu kartu alat musik. 2. Sistem menghitung jumlah alat yang sudah dipakai di keranjang ditambah estimasi pemakaian eksternal (Cek Ketersediaan Stok). 3. Jika stok masih mencukupi, sistem menambahkan satu unit alat ke objek Booking dan memperbarui label jumlah pada tombol keranjang. 4. Sistem menampilkan notifikasi singkat “<nama alat> ditambahkan”.
Alternate Flow	2a. Jika stok tidak mencukupi, sistem menolak penambahan dan menampilkan notifikasi “Stok <nama alat> tidak cukup!” tanpa mengubah isi keranjang.
Postcondition	Objek Booking pada CartManager memiliki entri instrument baru (atau qty bertambah) apabila stok mencukupi.

C. ACTIVITY DIAGRAM PROSES BISNIS UTAMA

Proses bisnis utama sistem adalah proses Checkout Pesanan, yaitu alur dari pelanggan memilih jadwal hingga pesanan tersimpan di database. Activity diagram berikut menggunakan tiga swimlane sesuai pihak yang bertanggung jawab: Pelanggan, Aplikasi RAFH Studio (Sistem), dan Database (SQLite). Diagram memuat initial node, aktivitas, dua titik percabangan (decision), alur alternatif pada masing-masing percabangan, serta final node.

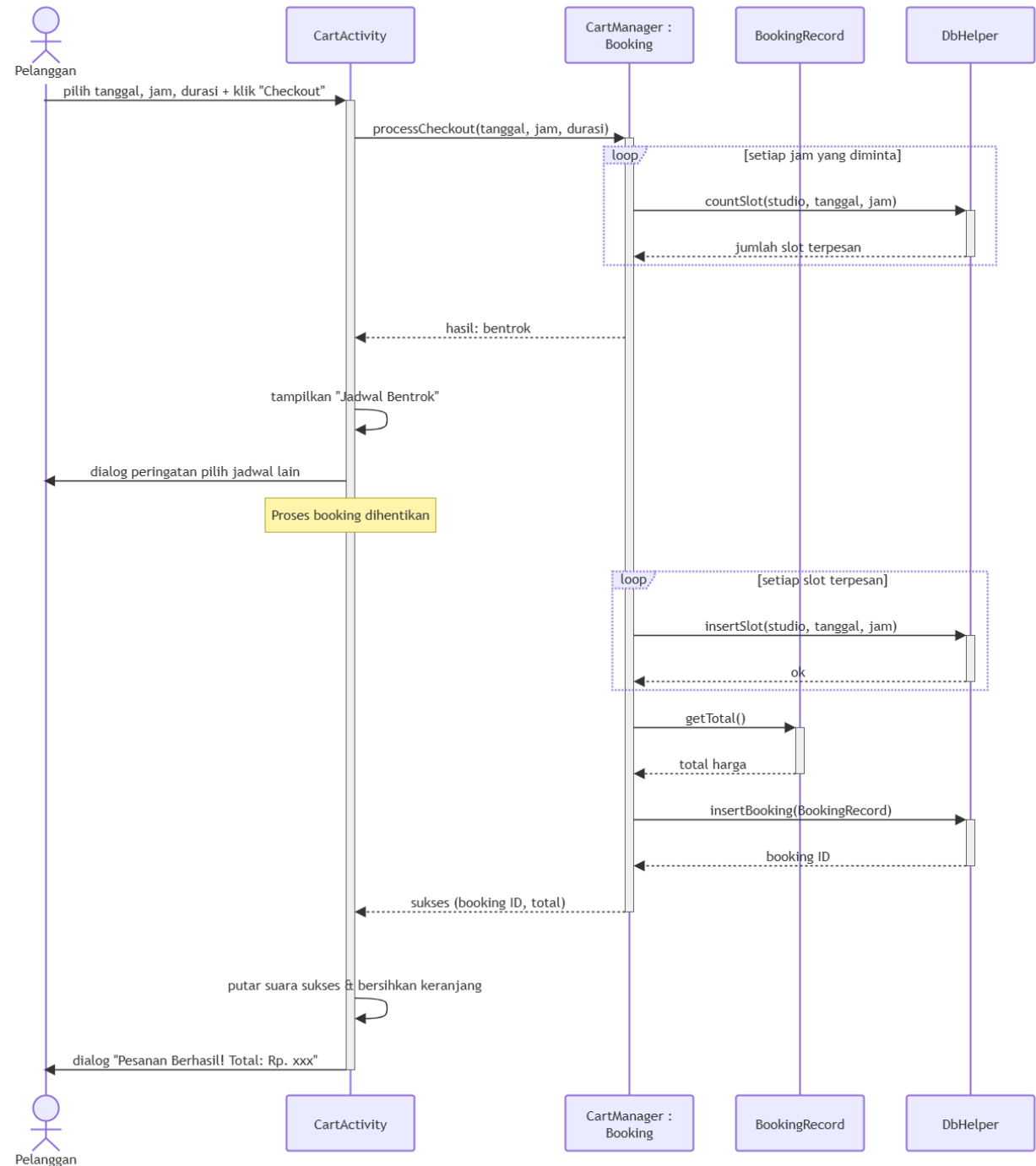


Gambar 3.1 Activity Diagram Proses Checkout Pesanan

Penjelasan singkat: proses dimulai ketika pelanggan mencari dan memilih studio/alat musik, kemudian menentukan tanggal, jam, dan durasi sewa. Saat tombol Checkout ditekan, sistem terlebih dahulu memeriksa apakah keranjang kosong; jika kosong, proses berhenti pada peringatan tanpa menyimpan data apa pun. Jika keranjang terisi, sistem memeriksa setiap slot jam yang diminta ke tabel `schedule_slots` pada database. Apabila ditemukan bentrok, sistem menampilkan peringatan dan mengembalikan alur ke pelanggan untuk memilih jadwal lain (alur alternatif). Apabila tidak ada bentrok, sistem mengunci slot jadwal dan menyimpan data pesanan ke tabel `bookings`, lalu menampilkan notifikasi keberhasilan kepada pelanggan hingga mencapai final node.

D. SEQUENCE DIAGRAM UNTUK USE CASE UTAMA

Sequence diagram berikut menggambarkan use case “Checkout Pesanan”, menampilkan actor (Pelanggan), boundary (CartActivity), control (CartManager yang mengelola objek Booking), entity (BookingRecord), dan database (DbHelper/SQLite), lengkap dengan urutan pesan dan kondisi alternatif (alt frame) ketika proses gagal karena bentrok jadwal.

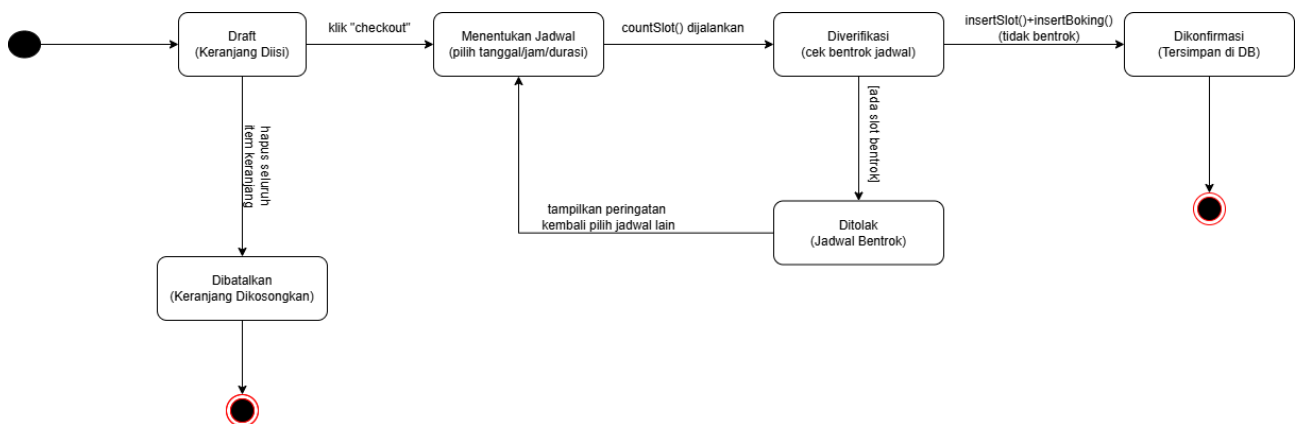


Gambar 4.1 Sequence Diagram Use Case “Checkout Pesanan”

Urutan pesan utama: (1) Pelanggan memilih jadwal dan menekan Checkout pada CartActivity; (2) CartActivity memanggil prosesCheckout() pada CartManager; (3) CartManager memanggil countSlot() ke DbHelper untuk setiap jam yang diminta; (4) DbHelper mengembalikan jumlah slot yang sudah terpakai. Pada blok alt [bentrok], CartManager langsung memberi tahu CartActivity untuk menampilkan dialog “Jadwal Bentrok!” kepada pelanggan dan proses dihentikan tanpa penyimpanan data. Pada blok alt [tidak bentrok], CartManager memanggil insertSlot() untuk mengunci jadwal, meminta getTotal() pada objek Booking, lalu memanggil insertBooking() pada DbHelper untuk menyimpan BookingRecord baru. Setelah tersimpan, CartActivity menampilkan dialog “Pesanan Berhasil” kepada pelanggan.

E. STATE-CHART DIAGRAM

Objek utama yang dipilih adalah Pesanan (Booking), karena objek ini mengalami beberapa kali perubahan status sejak pelanggan mulai mengisi keranjang hingga pesanan berhasil tersimpan atau dibatalkan. State-chart diagram berikut memuat enam status, initial state, dua kemungkinan final state, kejadian (event) yang memicu perubahan status, serta kondisi/guard pada percabangan status.



Gambar 5.1 State-Chart Diagram Objek “Pesanan (Booking)”

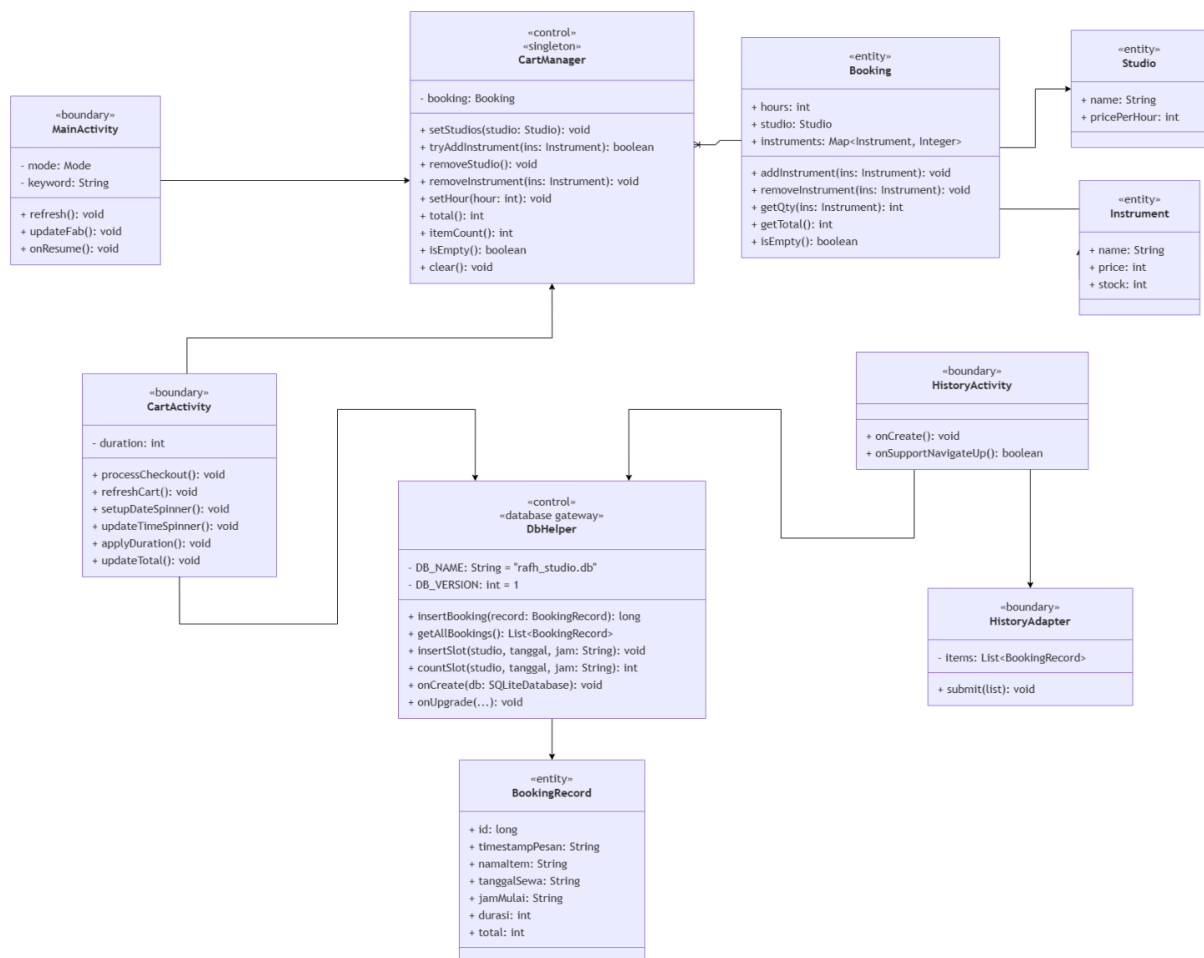
1) Penjelasan Status

- Draft (Keranjang Diisi) — status awal ketika pelanggan menambahkan studio dan/atau alat musik ke keranjang.
- MenentukanJadwal — dimasuki ketika pelanggan memilih tanggal, jam, dan durasi sewa; dapat berlanjut hanya jika keranjang tidak kosong.
- Diverifikasi — dimasuki saat pelanggan menekan tombol Checkout; sistem menjalankan countSlot() untuk memeriksa bentrok jadwal.

- Ditolak (Bentrok Jadwal) — dimasuki jika ditemukan slot yang sudah terpakai [guard: ada slot bentrok]; dari status ini alur kembali ke MenentukanJadwal agar pelanggan memilih jadwal lain.
- Dikonfirmasi (Tersimpan di DB) — dimasuki jika tidak ada bentrok [guard: tidak ada bentrok]; sistem menjalankan insertSlot() dan insertBooking(), lalu berlanjut ke final state “tersimpan”.
- Dibatalkan (Keranjang Dikosongkan) — status alternatif dari Draft apabila pelanggan menghapus seluruh item keranjang sebelum checkout, lalu berlanjut ke final state.

F. CLASS DESIGN

Class Diagram pada dokumen sebelumnya disempurnakan menjadi Class Design dengan menambahkan tipe data, visibility, method lengkap, relasi beserta multiplicity, serta penanda stereotype boundary/control/entity sesuai peran masing-masing class di dalam kode program aplikasi RAFH Studio.



Gambar 6.1 Class Design Sistem RAFH Studio

2) Ringkasan Class dan Peran (Stereotype)

- MainActivity — <<boundary>> — menampilkan katalog studio/alat musik & pencarian.
- CartActivity — <<boundary>> — mengelola tampilan keranjang, jadwal, dan proses checkout.
- HistoryActivity — <<boundary>> — menampilkan daftar riwayat pemesanan.
- HistoryAdapter / CardAdapter / CartAdapter — <<boundary>> — adapter RecyclerView penghubung data ke tampilan (HistoryAdapter digambarkan mewakili ketiganya pada diagram di atas).
- CartManager — <<control>> «singleton» — mengendalikan logika penambahan/penghapusan item dan validasi stok pada keranjang aktif.
- DbHelper — <<control>> «database gateway» — mengelola akses baca/tulis ke database SQLite lokal.
- Booking — <<entity>> — merepresentasikan satu transaksi sewa yang sedang disusun (studio, daftar alat musik, durasi).
- BookingRecord — <<entity>> — merepresentasikan satu baris riwayat transaksi yang sudah tersimpan di database.
- Studio — <<entity>> — data ruang studio beserta tarif per jam.
- Instrument — <<entity>> — data alat musik beserta harga sewa dan stok.

3) Relasi & Multiplicity

- CartManager 1 —◆ 1 Booking (komposisi): CartManager sebagai singleton selalu memiliki tepat satu objek Booking aktif yang siklus hidupnya bergantung pada CartManager.
- Booking 1 — 0..1 Studio: satu Booking dapat memiliki paling banyak satu Studio (boleh kosong jika pelanggan hanya menyewa alat musik).
- Booking 1 — 0..* Instrument: satu Booking dapat memuat nol atau banyak jenis Instrument, masing-masing dengan atribut jumlah (qty).
- DbHelper 1 — 0..* BookingRecord: satu DbHelper menyimpan dan membaca banyak baris BookingRecord dari tabel bookings.

- CartActivity, MainActivity, HistoryActivity masing-masing bergantung (dependency “uses”) pada CartManager dan/atau DbHelper untuk mengambil dan menyimpan data, tanpa memiliki (own) siklus hidup objek tersebut.

4) **Aturan Bisnis / Constraint (minimal tiga)**

- Ketersediaan stok: sebuah Instrument hanya boleh ditambahkan ke Booking apabila (jumlah pada keranjang + estimasi pemakaian eksternal) < stock; jika tidak, penambahan ditolak (constraint pada CartManager.tryAddInstrument()).
- Anti-bentrok jadwal: kombinasi (namaStudio, tanggal, jam) pada tabel schedule_slots bersifat unik (UNIQUE constraint) sehingga satu slot jam studio tidak dapat dipesan dua kali; pelanggaran aturan ini menghasilkan status Ditolak pada objek Booking.
- Batas durasi sewa: durasi sewa (dalam jam) dibatasi minimal 1 jam dan maksimal 24 jam per transaksi (constraint pada CartActivity, tombol tambah/kurang durasi).
- Keranjang tidak boleh kosong saat checkout: proses Checkout Pesanan hanya dapat dijalankan apabila Booking.isEmpty() bernilai false; jika keranjang kosong, sistem menampilkan peringatan dan tidak melanjutkan ke penyimpanan data.

Catatan: seluruh diagram pada dokumen ini disusun berdasarkan objek studi yang sama dengan proyek sebelumnya, yaitu aplikasi RAFH Studio, tanpa mengganti objek studi, sesuai ketentuan pengerjaan UAS.